Can LLMs Generalize to Structured Classification Tasks? A Comparative Analysis with Traditional Machine Learning Models in an Educational Context

Thien-Lac Quach[1,2][0009−0004−6310−5149] and Ton-Minh-Ky Tran[1,2][0009−0006−5027−2516]

[1] Faculty of Information Technology, University of Science, Ho Chi Minh City, Vietnam
[2] Vietnam National University, Ho Chi Minh City, Vietnam
{24125092,24125102}@apcs.fitus.edu.vn

Abstract. While Large Language Models (LLMs) excel in natural language processing, their ability to generalize to structured, non-linguistic tasks remains uncertain. This paper investigates this gap by comparing general-purpose LLMs and traditional machine learning models in a specific educational context–predicting students' final academic outcomes–using the Open University Learning Analytics Dataset (OULAD). To assess baseline generalization without task-specific adaptation, we deliberately exclude fine-tuning and retrieval-augmented generation. Instead, we evaluate two open-weight, small-size LLMs, Qwen-2.5-3B and LLaMA-3-8B, interpreting serialized JSON tabular data using zero-shot, one-shot, and five-shot prompting. These are benchmarked against seven hyperparameter-tuned traditional models, including Random Forest and Gradient Boosting. Our results demonstrate a severe performance gap: traditional ensemble methods significantly outperformed the LLMs, with the best-performing LLM peforming no better than chance. These findings highlight that current general-purpose LLMs struggle to interpret numerical feature vectors, making them currently unsuitable for complex and tabularized data compared to traditional classifiers. Future research should explore domain adaptation techniques or architectural modifications to bridge this modality gap.

Keywords: Large Language Models · Traditional Machine Learning Models · Classification problem · Generalization · Education

1 Introduction

Large Language Models (LLMs) have seen an unprecedented rise in popularity and application across various domains, including education. This integration of LLMs into education contexts has generated considerable interest, driven by their demonstrated proficiency in natural language understanding, text generation, and reasoning tasks [5]. However, while LLMs excel in natural language processing, one must pose the question: how well can these models generalize beyond language generation to perform effectively in structured, non-linguistic tasks, such as tabular classification? This question carries practical significance for Higher Education Institutions (HEIs), where the ability to predict students' academic outcomes from behavioral and demographic data underpins early-warning systems that identify at-risk students and allocate support resources.
Regarding classification tasks, there exists an extensive, well-established literature on the use of traditional machine learning models, such as Random Forest, Gradient Boosting, and Support Vector Machines, to address such problems. These models are explicitly trained on the target dataset, benefit from hyperparameter tuning, and operate natively on tabular feature vectors. Their effectiveness is well-documented [15, 22, 33, 7, 1]. LLMs, by contrast, acquire knowledge through large-scale pretraining on textual corpora and are not exposed to structured tabular data during this process. For instance, when presented with a student record serialized as JSON, the LLM must interpret a data format fundamentally different from the natural language it was trained on.

Recent studies have begun examining LLM performance on tabular data. Wen et al. (2025) proposed Tabular In-Context Learning (TabICL), a retrieval-augmented framework that adapts LLMs for tabular classification [30]. However, such approaches incorporate task-specific mechanisms (retrieval pipelines, structured prompting protocols) that blur the line between general-purpose inference and task-adapted learning. Furthermore, most existing comparisons operate on generic benchmarks (e.g., UCI, OpenML datasets) rather than domain-specific datasets with complex, multi-source feature engineering pipelines, a common characteristic of real-world educational data.

This gap motivates our central research question: To what extent can general-purpose LLMs, without any task-specific adaptation, generalize to a structured classification task, particularly in an educational context? We therefore deliberately exclude fine-tuning techniques (e.g., LoRA, QLoRA) and retrieval-augmented generation (RAG), as these modifications would transform the LLM into a task-adapted model, answering a different research question — namely, how much domain-specific adaptation is required to close the performance gap. To this end, we also favor simple prompting strategies (zero-shot, one-shot, few-shot) that do not require engineering complex prompt templates or retrieval mechanisms, as

these would again shift the focus from generalization to adaptation.

To this end, we evaluate two open-weight LLMs, Qwen-2.5-3B and LLaMA-3-8B, using zero-shot, one-shot, and five-shot prompting on the Open University Learning Analytics Dataset (OULAD) [23] and benchmark these against seven traditional machine learning models that undergo full hyperparameter tuning via cross-validation. We also designed a pipeline to serialize tabular data into a JSON format that is compatible with LLM input, ensuring that the evaluation reflects the LLMs' ability to interpret structured data rather than relying on engineered prompt templates. Our findings reveal that the tested LLMs underperform heavily compared to traditional models, with the best LLM configuration achieving a Macro F1 of 0.481, while the best traditional model reaches 0.722. This significant performance gap highlights the limitations of general-purpose LLMs in structured classification tasks and underscores the need for further research into domain adaptation techniques or architectural modifications to enhance their applicability in non-linguistic contexts.

The remainder of this paper is organized as follows. Section 2 provides background on the models used. Section 3 reviews related work on machine learning classifiers in education, data aggregation, and performance metrics. Section 4 describes the methodology, including the dataset, preprocessing pipeline, and experimental setup. Section 5 presents the results, and Section 6 discusses their implications, limitations, and directions for future work.

2 Background

2.1 Traditional Machine Learning Models

This paper uses seven traditional machine learning models for the classification task: Random Forest, Support Vector Machine, Gradient Boosting, Decision Tree, K-Nearest Neighbors, Extreme Learning Machine (ELM), and Multilayer Perceptron (MLP). The rationale for selecting these models is explained in the Methodology section. Below is a brief introduction to each model.

Random Forest Random Forest (RF) is an ensemble learning method first proposed by Ho in 1995 [18] and later developed by Breiman and Cutler [4]. The algorithm grows multiple decision trees and combines their outputs to produce a final prediction. Each tree is trained using a random vector Θ, generated independently from past vectors but with the same distribution [4]. The final prediction is obtained by aggregating ("voting") the predictions of all trees, typically via majority vote for classification [4]. RF is known for its robustness to overfitting, ability to handle high-dimensional data, and effectiveness in capturing complex interactions between features [4]. It remains one of the most effective machine learning models despite its early introduction [15].

Support Vector Machine Support Vector Machine (SVM) is a supervised learning model based on a simple idea: find an optimal hyperplane that separates the input data into different classes. The optimal hyperplane is defined as the one that maximizes the margin between classes, where the margin is the distance between the hyperplane and the nearest training points from each class (support vectors) [10]. SVM can be extended to handle non-linearly separable data by using kernel functions, particularly the Gaussian Radial Basis Function (RBF) kernel, which maps inputs into a higher-dimensional feature space where a linear separation is possible [26]. SVM is widely used for classification tasks due to its effectiveness in high-dimensional spaces and its ability to model complex relationships between features and classes [19].

Gradient Boosting Gradient Boosting (GB) is an ensemble learning method that builds a series of weak learners, typically trees, that learn from the residuals of previous learners. Residuals are the differences between the predicted values and the actual values of the target variable [9]. The training algorithm iterates through M boosting rounds. In each round m, a new weak learner $T_{(m)}$ is trained to predict the residuals, thereby rectifying mistakes made by previous learners [9]. The final prediction is obtained by summing the predictions of all weak learners, weighted by a learning rate ν that controls each learner's contribution [16]. The prediction process works as follows: for each tree $T_{(m)}$ and test instance x_i, the output $T_m(x_i)$ is scaled by ν and added to the cumulative prediction from previous trees, resulting in

$$F_M(x_i) = \sum_{m=1}^{M} \nu T_m(x_i).$$

GB is known for its high predictive accuracy and is widely used in various applications, including classification and regression tasks [16].

Decision Tree Decision Tree (DT) is a supervised learning model that recursively partitions the feature space into regions that are as homogeneous as possible with respect to the target label. At each internal node, the algorithm selects a feature (and a threshold for numerical features) that maximizes an impurity reduction criterion such as Gini impurity or information gain. Predictions are obtained by following the decision rules from the root to a leaf and outputting the majority class for classification tasks. Decision trees are interpretable and can capture non-linear decision boundaries, but they may overfit without appropriate regularization (e.g., maximum depth, minimum samples per split/leaf, or pruning).

K-Nearest Neighbors K-Nearest Neighbors (KNN) is a non-parametric, instance-based learning algorithm that classifies a data point based on the majority class among its k nearest neighbors in feature space [11]. Distance between data points is typically measured by Euclidean distance, although other measures such as Pearson or Spearman distance can also be used [3]. The choice of the

algorithm's sole hyperparameter, k, is crucial for performance. A larger value of k can make the model more robust to noise, though it may blur interclass boundaries [14]. Due to its simplicity, KNN's accuracy can be significantly affected by the choice of k and the nature of the data, and it often requires feature scaling to ensure that all features contribute equally to the distance calculations [24]. KNN is noted for its strong consistency: as the dataset size increases, the probability of KNN making an error converges to no worse than twice the Bayes error rate, which is the lowest possible error rate for any classifier [11].

Extreme Learning Machine Extreme Learning Machine (ELM) is a type of feedforward neural network, first proposed by Huang et al. in 2006 [21]. ELM is a learning algorithm for single-hidden layer feedforward neural networks (SLFNs) whose training is significantly faster than traditional counterparts. ELM randomly assigns input weights and hidden-layer biases, and then analytically determines the output weights by solving a linear system of equations [21]. The training process consists of three steps: (1) randomly generate the input weights and hidden-layer biases; (2) compute the hidden-layer output matrix; (3) calculate the output weights using the Moore–Penrose generalized inverse of the hidden-layer output matrix [21]. ELM has been shown to achieve good generalization performance with significantly reduced training time compared to traditional feedforward neural networks, making it suitable for large-scale classification tasks [20].

Multilayer Perceptron Multilayer Perceptron (MLP) is a class of feedforward artificial neural network that consists of at least three layers: an input layer, one or more hidden layers, and an output layer [25]. First proposed by Rosenblatt in 1958, MLP also serves as a basis for more complex neural network architectures, upon which ELM is built. Crucial in MLP architecture are the concepts of units and hidden layers. Units, also known as neurons, are the basic computational elements of MLP that receive input, process it, and produce output. Hidden layers are the layers of units that exist between the input and output layers, allowing MLP to learn complex representations of data [25]. MLP functions by assigning weights to the connections between units and applying an activation function (commonly a sigmoid or ReLU function) to the output of each neuron. Learning in MLP is typically achieved through backpropagation and gradient descent algorithms, which adjust the weights to minimize the error between the predicted and actual outputs (modeled by a loss function) [17].

2.2 Large Language Models

Large Language Models (LLMs) present a significant advancement in artificial intelligence, particularly in the field of natural language processing (NLP). The architecture of LLMs is based on the Transformer model, first introduced in a foundational paper by Vaswani et al. in 2017 [29]. The Transformer architecture utilizes self-attention mechanisms to process input data, allowing it to capture long-range

dependencies and contextual information effectively [29]. LLMs are trained on enormous amounts of data and often possess billions of parameters, enabling them to generate human-like text and perform a wide range of language tasks with high accuracy [5]. The training process of LLMs typically involves two stages: pre-training and fine-tuning. During pre-training, the model learns general language patterns from a large corpus of text, while fine-tuning involves further training the model on specific tasks or domains to enhance its performance in those areas [12]. LLMs have revolutionized the field in that they can generalize across a wide range of tasks without task-specific training, and they have been shown to outperform traditional machine learning models in various NLP tasks [5].

For our paper, we choose two models: **Qwen-2.5-3B** and **LLaMA-3-8B** (the suffix XB of each model's name refers to the number of billions of parameters X). These models are publicly available on Hugging Face. Below is a brief introduction to each model.

Qwen-2.5-3B Qwen-2.5-3B is a large language model developed by Alibaba, which is part of the Qwen series of models. The Qwen 2.5 series is designed to be more efficient than its predecessors, especially on coding and mathematical tasks. It has also been reported to enhance instruction-following, producing longer responses (beyond 8K tokens), interpreting structured inputs (e.g., tables), and generating structured outputs, particularly in JSON format. It is also more robust to variations in system prompts, which improves role-playing behavior and condition control in chatbot applications.

LLaMA-3-8B LLaMA-3-8B is a large language model developed by Meta as part of the LLaMA series. It is designed to be efficient and capable across diverse language tasks. Architecturally, LLaMA 3 is an autoregressive language model built on an optimized Transformer backbone. Its tuned variants are aligned with human preferences for helpfulness and safety through supervised fine-tuning (SFT), followed by reinforcement learning from human feedback (RLHF).

3 Related Work

3.1 Machine Learning Models as Classifiers in Educational Context

The use of machine learning models as classifiers in educational contexts has been explored in various studies [22, 33, 28, 1]. From these studies, two main conclusions can be drawn. First, certain models often outperform others in terms of accuracy for general classification tasks. Most prominently, ensemble methods—particularly Random Forest (RF)—consistently rank as top performers: Delgado et al. (2014) established RF as a "gold standard" across 179 classifiers [15], a finding echoed in educational contexts by Jimenez-Macias et al. (2025) [22]. Support Vector Machines and Artificial Neural Networks also prove effective for capturing non-linear relationships in educational data, with

reported accuracies reaching 97.88% and 99%, respectively [33, 1]. Gradient Boosting variants such as XGBoost have also shown similar strength in identifying at-risk students [1, 15]. More recently, stacking ensemble models have been explored; these methods train heterogeneous base learners (e.g., KNN, Naive Bayes, RF, and XGBoost) and aggregate their outputs using a meta-learner such as Logistic Regression to mitigate individual model biases. For instance, a stacking framework achieved an accuracy of 98.53% in predicting learning achievements on the XuetangX MOOC dataset [28].

Second, a critical consensus across current literature is the superiority of behavioral engagement data over demographic information for classification tasks. Online behavioral logs, including video watch counts, forum posts, and quiz participation, are significantly more influential in predicting academic risk than static factors like age, gender, or educational background. Jimenez-Macias et al. (2025) further refined this by demonstrating that time spent on tasks is a more sensitive predictor of performance than the raw number of attempts [22]. Moreover, the distribution of grades across interactions significantly impacts model accuracy, as uniform grade distributions enhance the algorithm's ability to generalize patterns and make accurate predictions [22].

Methodological integrity in this context requires addressing class imbalance and interpretability. Educational datasets are frequently imbalanced, often containing significantly fewer failing students than passing ones [7, 28]. To address this, researchers employed data-level solutions like SMOTE (Synthetic Minority Oversampling Technique) or Sequential Conditional GANs to balance class distributions and prevent model bias toward the majority class [7]. Furthermore, as models grow in complexity, tools like SHAP (SHapley Additive exPlanations) are increasingly used to provide "white-box" transparency, identifying which specific student behaviors drive the final prediction [28].
Despite these advancements, inconsistent preprocessing and feature scaling remain primary challenges, as they are often cited as the reason for varied performance reports for identical models across different studies [1]. Finally, the emerging paradigm of Tabular In-Context Learning (TabICL) using Large Language Models (LLMs) offers a novel interface for classification that can complement traditional frameworks like XGBoost and CatBoost, which have served as state-of-the-art benchmarks for tabular data for decades [30].

3.2 Data aggregation

It stands to reason that gathering high-quality and representative data is often a difficult endeavor [2]. Alias et al. (2024) delineated a framework in which a list of hypothetically relevant features is enumerated and filtered via information gain. More specifically, the features taken into consideration include those relating to students' study hours, participation in extracurricular activities, hours of sleep, and mock exams practiced [27]. Surveys for these features are

deployed online as a questionnaire using the platform Google Forms.

3.3 Performance metrics

Common metrics to assess classification algorithms include accuracy, precision, recall, F1-score [2, 32], and the area under the receiver operating characteristic curve (AUC-ROC) [6, 31]. These metrics provide insights into a model's ability to correctly classify instances, handle imbalanced datasets, and make informed decisions based on predictions [6]. Techniques for handling imbalanced datasets are crucial when working with decision tree (DT)-based models to avoid bias toward the majority class. Here, we focus on the Synthetic Minority Over-sampling Technique (SMOTE), specifically its variant ADASYN, which utilizes a weighted distribution to determine the learning difficulty of minority-class samples and generates synthetic data accordingly [8, 13]. This approach helps to balance the dataset and improve the performance of decision tree models in predicting the minority class.

4 Methodology

In this section, we explain the methodology used to conduct our research. We first describe the dataset and preprocessing pipeline. Next, we outline the traditional machine learning models and LLMs we employed for our experiments. Finally, we discuss the evaluation metrics and experimental setup.

4.1 Dataset and Preprocessing

We utilized the Open University Learning Analytics Dataset (OULAD), described in [23]. This dataset reports the background, behavior, and performance of students in England within a Virtual Learning Environment (VLE). The dataset contains 32,593 records with 20 features, including demographic information, course interactions, and final results. The features include both categorical and numerical data, which required preprocessing before model training. As outlined by its authors, the dataset consists of four modules:

Student Demographics: This module includes background information about the students, such as age, gender, and educational as well as socio-economic background. One notable feature is the Index of multiple deprivation (IMD), which is a measure of socio-economic status based on the students' postal codes.

Course Registrations: This module contains information about the courses students are registered for, including course/presentation codes and their lengths. It also includes data on students' registration and unregistration dates.

Assessment Results: This module captures the students' performance in three types of assessments, namely Tutor Marked Assessments (TMA), Computer Marked Assessments (CMA), and Final Exams (Exam), adjusted by weights. It provides insights into their academic progress and outcomes.

VLE interactions: This module records the students' interactions with the Virtual Learning Environment (VLE), including the number of clicks on various pages and resources. It also includes the dates of these interactions, which can be used to analyze engagement patterns over time. This module also provides information on the resources in the VLE.

Module	Table	Records	What it contains (main fields)
Student Demographics	`studentInfo`	32,593	Student profile and outcome labels: `id_student`, demographics (gender, region, education, disability), socio-economic and age bands, prior attempts, studied credits, and `final_result`.
Course Registrations	`courses`	22	Module-presentation metadata: `code_module`, `code_presentation`, and course `length`.
Registrations	`studentRegistration`	32,593	Enrollment timeline per student and presentation: `id_student`, `date_registration`, and `date_unregistration`.

Table 1: OULAD: Student and Course Demographics

Module	Table	Records	What it contains (main fields)
Assessment Results	assessments	206	Assessment structure: `code_module`, `code_presentation`, `id_student`, `id_assessment`, `assessment_type`, `date`, `weight`
Assessment Results	studentAssessment	173,912	Student performance: `id_student`, `id_assessment`, `is_banked`, `date_submitted`, `score`

Table 2: OULAD: Assessment Data

Module	Table	Records	What it contains (main fields)
VLE interactions	interactions	10,655,280	Clickstream data: `code_module`, `code_presentation`, `id_student`, `id_site`, `date`, `sum_click`
VLE interactions	vle	6,364	VLE resource metadata: `id_site`, `code_module`, `code_presentation`, `activity_type`, `week_from`, `week_to`

Table 3: OULAD: Virtual Learning Environment Interactions

To correctly and effectively classify the students' final results, preprocessing must be done to merge the tables and create a single table containing all the features, keyed according to the `id_student` column. In the next subsection, we describe the preprocessing steps taken to achieve this.

We divide the tables into three groups: background (studentInfo), performance (studentAssessment and assessments), and behavior (studentVle, vle, and assessments for cutoff construction). We perform the following preprocessing tasks for each group:

Background: We first perform one-hot encoding on the categorical features in the `studentInfo` table, which include students' gender, region, highest education and disability status. For age and IMD bands, we perform ordinal encoding, as these features have an inherent order.

Performance: We first merge the table `studentAssessment` and the table `assessments` on `id_assessment`. Missing due dates and scores are explicitly tracked using binary indicators (`has_due_date` and

`score_missing`), and missing values are imputed using the median. We then engineer timing features from submission behavior, including `submission_delay`, late/early flags, and a capped delay variable based on the 1st and 99th percentiles to reduce outlier effects. In addition, we compute weighted performance (`weighted_score`) using assessment weights and its min-max normalized version. Finally, we aggregate all assessment records at the student-module-presentation level (`id_student`, `code_module`, `code_presentation`) to produce summary statistics (counts, central tendency, variability, weighted-score totals, lateness ratios, delay average, and missingness ratios), and append assessment-type frequency features via a pivot table.

Behavior: To construct behavior features from VLE interactions, we first derive a module-presentation cutoff date as the latest assessment date and keep only clicks that occur on or before this cutoff, preventing post-assessment leakage. We then merge `studentVle` with `vle` to attach `activity_type` for each `id_site` (with missing types labeled as unknown). Next, we aggregate `sum_click` values per student-module-presentation and activity type, and pivot them into wide features (e.g., `clicks_forum`, `clicks_resource`). Because interactions are processed in chunks for scalability, we finally sum partial chunk-level results to obtain one behavior-feature row per (`id_student`, `code_module`, `code_presentation`).

After preprocessing, we aggregate the three groups of features using `id_student` as the key to create the final dataset for modeling. This dataset contains 32,593 records and 72 features, including the students' ID and their final results. The final results are categorized into four classes: `Distinction`, `Pass`, `Fail`, and `Withdrawn`.

Next, we split the dataset into training and testing sets, with an 80–20 split. We use `sklearn`'s `SMOTE` to address class imbalance in the training set, which is a common issue in educational datasets where certain outcomes (e.g., withdrawal) may be more prevalent than others. This increases the size of the training set to 39,556 records, which raises the total dataset size to 46,075 records.

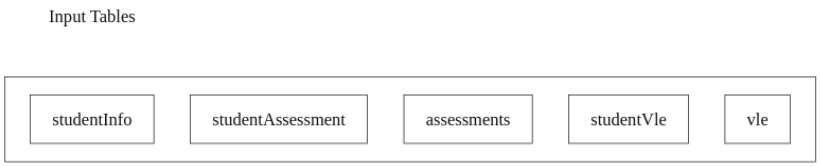

Fig. 1: Preprocessing pipeline (Stage 1): input tables.

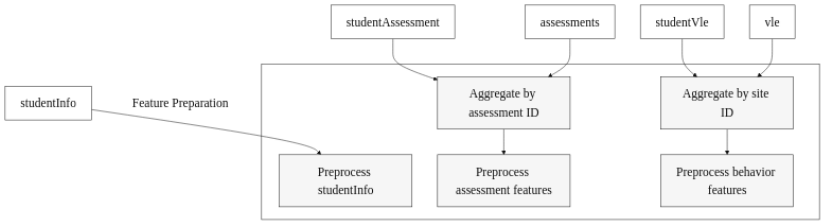

Fig. 2: Preprocessing pipeline (Stage 2): feature preparation.

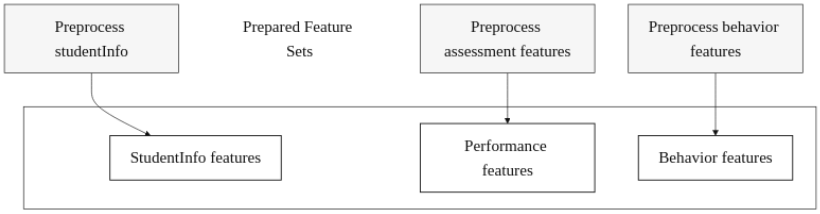

Fig. 3: Preprocessing pipeline (Stage 3): prepared feature sets.

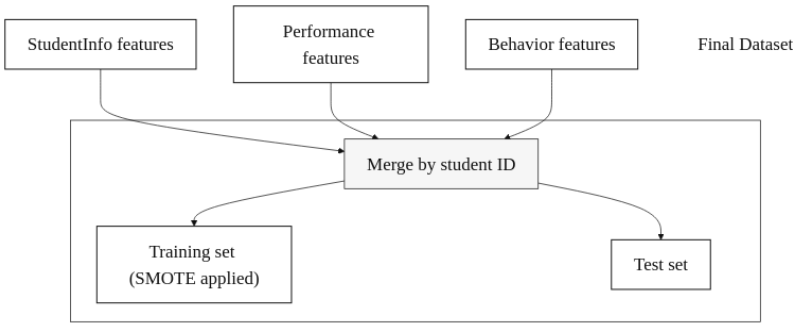

Fig. 4: Preprocessing pipeline (Stage 4): final dataset construction.

4.2 Modeling

Traditional Machine Learning Models In our paper, we utilize seven traditional machine learning models as baselines for comparison with large language models. These models include: Random Forest (RF), Support Vector Machine (SVM), Gradient Boosting (GB), Decision Tree (DT), K-Nearest Neighbors (KNN), Extreme Learning Machine (ELM), and Multilayer Perceptron (MLP).

The first two models, RF and SVM, are learning methods identified as top classifiers in Delgado et al.'s systematic review of educational data mining [15]. The paper also highlights other strong models, such as DT (C5.0 implementation), GB, ELM, and MLP. A review of the OULAD dataset by Alhakbani and Alnassar [1] also identifies DT (CART implementation) as a top-performing model in detecting at-risk students. Finally, KNN is also included, due to its simplicity, as we intend to explore if the classification task can be effectively solved using a simple distance-based method.

Regarding implementation, all models are run using `sklearn` libraries; for the DT model, we also use the R programming language to implement the C5.0 algorithm, since `sklearn` only provides the CART implementation. We first run a 5-fold cross-validation on the training set to perform hyperparameter tuning for each model. We then select the best hyperparameters based on the average Macro F1 across folds and train the final model on the entire training set using these hyperparameters.

Large Language Models For baseline LLMs, we utilize two models available on Hugging Face: `Qwen-2.5-3B` and `LLaMA-3-8B`. The models are not fine-tuned and were prompted to perform the classification task using zero-shot, one-shot, and five-shot prompting.

Model	Hyperparameters
Random Forest	`n_estimators=300, min_samples_split=5, min_samples_leaf=2, max_features='sqrt', max_depth=50, bootstrap=False`
Decision Tree	`splitter='best', min_samples_split=20, min_samples_leaf=4, max_features=None, max_depth=10, criterion='gini'`
Support Vector Machine	`StandardScaler() + SVC(C=0.7847599703514611, class_weight='balanced', kernel='rbf', gamma='auto')`
Gradient Boosting	`subsample=1.0, n_estimators=100, min_samples_split=2, min_samples_leaf=4, max_features=None, max_depth=5, learning_rate=0.2`
K-NN	`StandardScaler() + KNeighborsClassifier (n_neighbors=7, weights='uniform', p=1, metric='minkowski', leaf_size=50)`
Extreme Learning Machine	`ELMClassifier(n_neurons=1000, alpha=0.0001, ufunc='relu')`
Multilayer Perceptron	`MLPClassifier(hidden_layer_sizes=(32,), activation='tanh', alpha=0.1, solver='lbfgs', max_iter=1500, max_fun=15000)`

Table 4: Hyperparameters used for each traditional machine learning model.

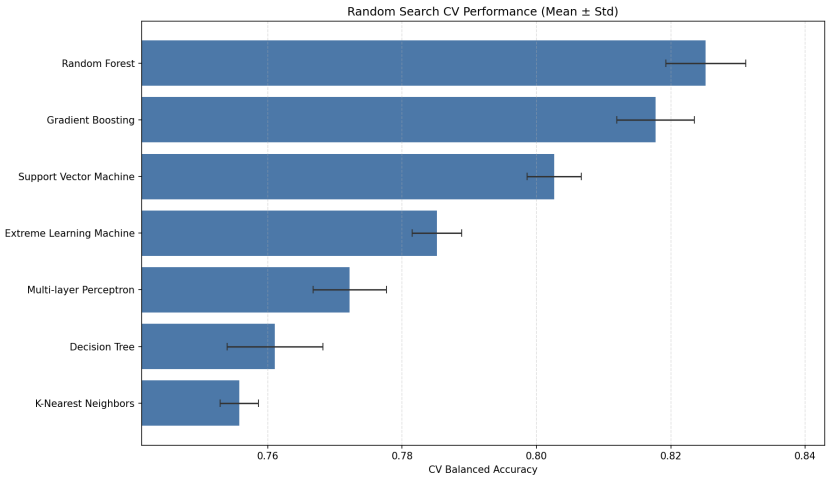

Fig. 5: Cross-validation results for traditional machine learning models.

Listing 1.1: Function to build LLMs prompt

```
def build_prompt(
    base_prompt: str,
    row_dict: dict,
    few_shot_examples: list[tuple[dict, str]],
) -> str:
    current_row_json = json.dumps(row_dict, ensure_ascii=False)

    parts = [
        base_prompt,
        "Allowed labels: Distinction, Pass, Withdrawn, Fail.",
        "Infer the label from the pattern in the examples.",
        "Do not default to the most common example label.",
        "Output exactly one line in this format:",
        "LABEL=<one of Distinction, Pass, Withdrawn, Fail>",
        "Do not output anything else.",
    ]

    if few_shot_examples:
        parts.append("")
        parts.append("Examples:")

        for idx, (example_x, example_y) in enumerate(few_shot_examples,
    start=1):
            example_x_json = json.dumps(sanitize_row(example_x),
    ensure_ascii=False)
            parts.extend(
                [
                    f"Example {idx}:",
                    f"Student record:\n{example_x_json}",
                    f"LABEL={example_y}",
                    "",
                ]
            )

    parts.extend(
        [
            "Now classify this record:",
            f"Student record:\n{current_row_json}",
            "Answer:",
        ]
    )

    return "\n".join(parts)
```

5 Results

Model	Technique	Macro F1	Balanced Acc.
Traditional ML			
Decision Tree	–	0.675	0.685
Random Forest	–	0.720	0.719
Gradient Boosting	–	0.722	0.718
Support Vector Machine	–	0.652	0.635
K-Nearest Neighbors	–	0.601	0.595
MLP Classifier	–	0.678	0.676
ELM Classifier	–	0.645	0.630
LLMs			
Qwen-2.5-3B	0-shot	0.218	0.297
Qwen-2.5-3B	1-shot	0.387	0.562
Qwen-2.5-3B	5-shot	0.306	0.377
LLaMA-3-8B	0-shot	0.280	0.341
LLaMA-3-8B	1-shot	0.392	0.509
LLaMA-3-8B	5-shot	0.481	0.494

Table 5: Summary of traditional ML and LLM performance on the test set (Macro F1 and Balanced Accuracy).

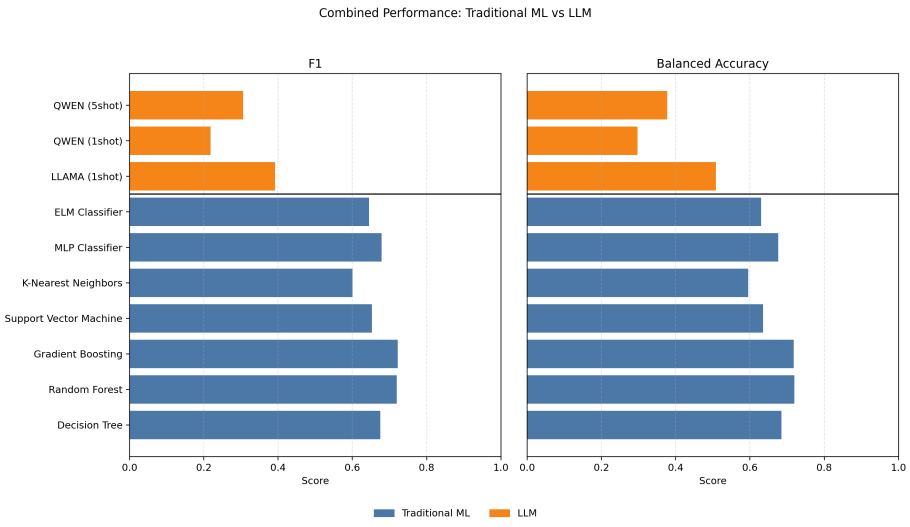

Fig. 6: Combined performance comparison of traditional machine learning models and large language models using F1 and Balanced Accuracy.

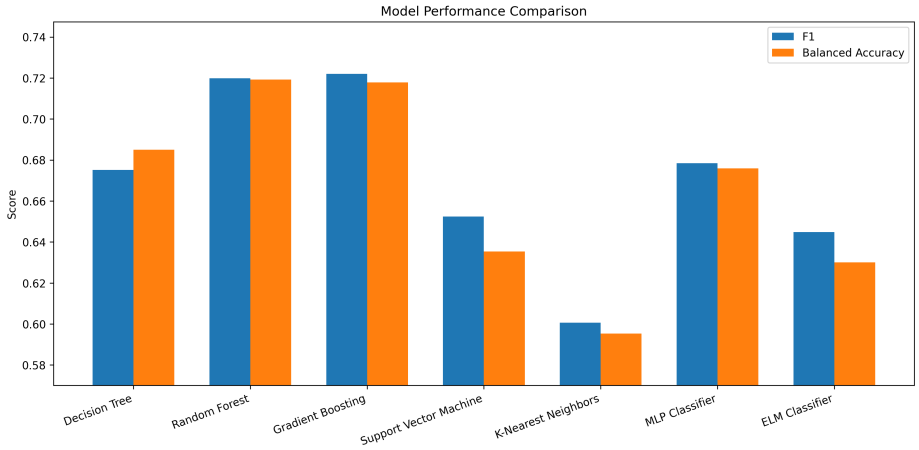

Fig. 7: Test-set performance comparison of traditional predictive models using Macro F1 and Balanced Accuracy.

5.1 Traditional Predictive Models

Various traditional model architectures are implemented. The evaluated models are Random Forest, Decision Tree, Support Vector Machine, Gradient Boosting, K-Nearest Neighbors, Extreme Learning Machine, and Multilayer Perceptron. The performance of each model is measured using plain accuracy, balanced accuracy, and F1 score. The accuracy of a model can be calculated as

$$\frac{\text{True Positive} + \text{True Negative}}{\text{Total Predictions}}.$$

It is an intuitive measure of how correct the model predictions are, but falls short when the given dataset is imbalanced. To mitigate this issue, the balanced accuracy is preferred:

$$\frac{\text{Sensitivity} + \text{Specificity}}{2}.$$

In addition, the F1-score is also utilized, calculated as the harmonic mean of precision and recall:

$$\frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

As shown in Fig. 7, the model performance plot summarizes the comparative results across all evaluated traditional predictive models. Ensemble methods remain the top performers, with Gradient Boosting and Random Forest achieving the highest Macro F1 scores of 0.722 and 0.720, respectively. Support Vector Machine and Extreme Learning Machine also show competitive performance, while K-Nearest Neighbors has the lowest Macro F1 among the traditional models at 0.601. These results are consistent with existing literature that highlights the effectiveness of ensemble methods in classification tasks, particularly in educational contexts where complex interactions between features are common.

5.2 Large Language Models

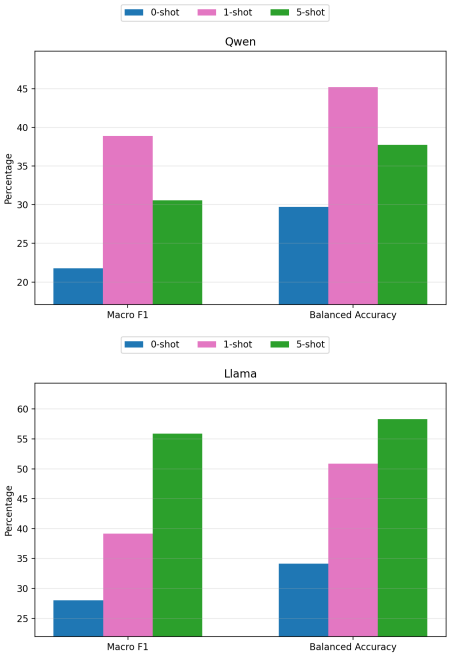

Fig. 8: Prompting-technique comparison for Qwen-2.5-3B (left) and LLaMA-3-8B (right), using Macro F1 and Balanced Accuracy.

We report Macro F1 for LLMs. Macro F1 treats each class as equally important by averaging the per-class F1 scores. This is particularly useful in our case, where the dataset is imbalanced, and we want to ensure that performance on minority classes (e.g., at-risk students) is not overshadowed by performance on majority classes (e.g., passing students). As shown in Fig. 8, the best-performing LLM configuration is LLaMA-3-8B with 5-shot prompting, which achieves a Macro F1 of 0.481 and a balanced accuracy of 0.494. Qwen-2.5-3B performs best with 1-shot prompting (Macro F1 = 0.387; balanced accuracy = 0.562). Overall, Macro F1 values range from 0.218 to 0.481 and balanced accuracies range from 0.297 to 0.562.

Even the best-performing LLM configuration (LLaMA-3-8B with 5-shot prompting) achieves a Macro F1 of only 0.481, which is lower than the lowest-scoring traditional model (KNN with a Macro F1 of 0.601). This suggests that the bottleneck is not model capacity—LLaMA-3-8B has 8 billion parameters compared to KNN's zero learned parameters—but rather the mismatch between the LLM pretraining distribution (natural language) and the input modality (serialized numerical feature vectors).

This gap also contextualizes the practical implications for HEIs: at current scales, deploying general-purpose LLMs for student outcome classification without fine-tuning would produce predictions no better than a majority-class baseline, rendering them unsuitable for early-warning systems where per-class sensitivity is critical.

6 Discussion

6.1 Interpretation

The results highlight the strong performance of traditional machine learning models, particularly Gradient Boosting. In particular, Gradient Boosting predicts students' final results with a balanced accuracy of 0.718. This suggests that students' access to study materials, as well as study time, can strongly affect academic outcomes.

On the other hand, LLMs show consistently weaker performance, with balanced accuracy ranging from 0.297 to 0.562 and Macro F1 ranging from

0.218 to 0.481. One contributing factor is that LLMs can fall back on default heuristics (most commonly predicting "Fail" or "Pass") when the input is a serialized feature vector rather than natural language. In our experiments, one-shot prompting yields the best balanced accuracy for both models, while five-shot prompting improves Macro F1 for LLaMA.

From a pedagogical standpoint, our study serves as a rudimentary framework for implementing an early-warning system for students. This supports the role of traditional machine learning models for this purpose and illustrates the limitations of general-purpose LLMs in structured classification problems.

6.2 Threats to Validity

Both approaches rely on OULAD, which lacks access to external factors that may affect students' academic performance. This includes, but is not limited to, student mental health and sudden emergencies, which may act as black swan events that reduce predictive accuracy.

There is a risk of models overfitting to historical cohorts, which may reinforce biases that do not generalize to more recent educational settings. Furthermore, students who are aware that their activities are monitored may exhibit different study patterns than those who are not, which can change the models' prediction accuracy.

7 Conclusion

This study presented a comparative analysis of traditional machine learning models and Large Language Models for predicting students' final outcomes based on their footprint in an online learning system and engagement patterns. The results show substantially better performance for traditional models than for general-purpose LLMs, reinforcing the effectiveness of Random Forest and Gradient Boosting as strong baselines for structured classification problems.

Our future work aims to refine the prompts fed to the LLMs for more accurate predictions, as well as evaluate larger or more capable LLMs than those used in this study. Furthermore, additional socio-economic data and more diverse datasets across disciplines would aid our analysis of the generalizability of engagement features.

References

1. Alhakbani, H.A., Alnassar, F.M.: Open Learning Analytics: A Systematic Review of Benchmark Studies using Open University Learning Analytics Dataset (OULAD). In: 2022 7th International Conference on Machine Learning Technologies (ICMLT). ICMLT 2022, pp. 81–86. ACM (2022). https://doi.org/10.1145/3529399.3529413
2. Alias, M.A.H., Abdul Aziz, M.A., Hambali, N., Taib, M.N.: Student performance classification: a comparison of feature selection methods based on online learning activities. International Journal of Electrical and Computer Engineering (IJECE) **14**(4), 4675 (2024). https://doi.org/10.11591/ijece.v14i4.pp4675-4685
3. Andretta Jaskowiak, P., Campello, R.: Comparing Correlation Coefficients as Dissimilarity Measures for Cancer Classification in Gene Expression Data. In: (2011)

4. Breiman, L.: Random Forests. Machine Learning **45**(1), 5–32 (2001). https://doi.org/10.1023/a:1010933404324
5. Brown, T.B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D.M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., Amodei, D.: Language Models are Few-Shot Learners, (2020). https://doi.org/10.48550/ARXIV.2005.14165.
6. Brzezinski, D., Stefanowski, J.: Prequential AUC: properties of the area under the ROC curve for data streams with concept drift. Knowledge and Information Systems **52**(2), 531–562 (2017). https://doi.org/10.1007/s10115-017-1022-8
7. Bujang, S.D.A., Selamat, A., Ibrahim, R., Krejcar, O., Herrera-Viedma, E., Fujita, H., Ghani, N.A.M.: Multiclass Prediction Model for Student Grade Prediction Using Machine Learning. IEEE Access **9**, 95608–95621 (2021). https://doi.org/10.1109/access.2021.3093563
8. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic Minority Over-sampling Technique. (2011). https://doi.org/10.48550/ARXIV.1106.1813
9. Chen, H.-H.: Understanding Gradient Boosting Classifier: Training, Prediction, and the Role of γ_j. (2024). https://doi.org/10.48550/ARXIV.2410.05623. arXiv: 2410.05623 [cs.LG]
10. Cortes, C., Vapnik, V.: Support-vector networks. Machine Learning **20**(3), 273–297 (1995). https://doi.org/10.1007/bf00994018
11. Cover, T., Hart, P.: Nearest neighbor pattern classification. IEEE Transactions on Information Theory **13**(1), 21–27 (1967). https://doi.org/10.1109/tit.1967.1053964
12. Devlin, J., Chang, M.-W., Lee, K., Toutanova, K.: BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, (2018). https://doi.org/10.48550/ARXIV.1810.04805.
13. Elreedy, D., Atiya, A.F.: A Comprehensive Analysis of Synthetic Minority Oversampling Technique (SMOTE) for handling class imbalance. Information Sciences **505**, 32–64 (2019). https://doi.org/10.1016/j.ins.2019.07.070
14. Everitt, B.S., Landau, S., Leese, M., Stahl, D.: Cluster Analysis. Wiley (2011)
15. Fernández-Delgado, M., Cernadas, E., Barro, S., Amorim, D.: Do we Need Hundreds of Classifiers to Solve Real World Classification Problems? Journal of Machine Learning Research **15**(90), 3133–3181 (2014). http://jmlr.org/papers/v15/delgado14a.html
16. Friedman, J.H.: Greedy function approximation: A gradient boosting machine. The Annals of Statistics **29**(5) (2001). https://doi.org/10.1214/aos/1013203451
17. Haykin, S.S.: Neural networks. A comprehensive foundation. Prentice Hall, Upper Saddle River, NJ (1999)
18. Ho, T.K.: Random decision forests. In: Proceedings of 3rd International Conference on Document Analysis and Recognition. ICDAR-95, pp. 278–282. IEEE Comput. Soc. Press. https://doi.org/10.1109/icdar.1995.598994
19. Hsu, C.-w., Chang, C.-c., Lin, C.-J.: A Practical Guide to Support Vector Classification Chih-Wei Hsu, Chih-Chung Chang, and Chih-Jen Lin. (2003)
20. Huang, G.-B., Zhou, H., Ding, X., Zhang, R.: Extreme Learning Machine for Regression and Multiclass Classification. IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics) **42**(2), 513–529 (2012). https://doi.org/10.1109/tsmcb.2011.2168604

21. Huang, G.-B., Zhu, Q.-Y., Siew, C.-K.: Extreme learning machine: Theory and applications. Neurocomputing **70**(1–3), 489–501 (2006). https://doi.org/10.1016/j.neucom.2005.12.126
22. Jiménez-Macías, A., Muñoz-Merino, P.J., Moreno-Marcos, P.M., Delgado Kloos, C.: Evaluation of traditional machine learning algorithms for featuring educational exercises. Applied Intelligence **55**, 501 (2025). https://doi.org/10.1007/s10489-025-06386-5
23. Kuzilek, J., Hlosta, M., Zdrahal, Z.: Open University Learning Analytics dataset. Scientific Data **4**(1) (2017). https://doi.org/10.1038/sdata.2017.171
24. Nigsch, F., Bender, A., van Buuren, B., Tissen, J., Nigsch, E., Mitchell, J.B.O.: Melting Point Prediction Employing k-Nearest Neighbor Algorithms and Genetic Parameter Optimization. Journal of Chemical Information and Modeling **46**(6), 2412–2422 (2006). https://doi.org/10.1021/ci060149f
25. Rosenblatt, F.: The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review **65**(6), 386–408 (1958). https://doi.org/10.1037/h0042519
26. Schölkopf, B., Smola, A.J.: Learning with Kernels: support vector machines, regularization, optimization, and beyond. MIT Press (2002)
27. Suleiman, I.B., Okunade, O.A., Dada, E.G., Ezeanya, U.C.: Key factors influencing students' academic performance. Journal of Electrical Systems and Information Technology **11**(1) (2024). https://doi.org/10.1186/s43067-024-00166-w
28. Tong, T., Li, Z.: Predicting learning achievement using ensemble learning with result explanation. PLOS ONE **20**(1), e0312124 (2025). https://doi.org/10.1371/journal.pone.0312124
29. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need, (2017). https://doi.org/10.48550/ARXIV.1706.03762.
30. Wen, X., Zheng, S., Xu, Z., Sun, Y., Bian, J.: Scalable In-Context Learning on Tabular Data via Retrieval-Augmented Large Language Models. Preprint (2025)
31. Yang, Z., Zhang, T., Lu, J., Zhang, D., Kalui, D.: Optimizing area under the ROC curve via extreme learning machines. Knowledge-Based Systems **130**, 74–89 (2017). https://doi.org/10.1016/j.knosys.2017.05.013
32. Zaky, U., Naswin, A., Sumiyatun, S., Murdiyanto, A.W.: Performance Analysis of the Decision Tree Classification Algorithm on the Water Quality and Potability Dataset. Indonesian Journal of Data and Science **4**(3), 145–150 (2023). https://doi.org/10.56705/ijodas.v4i3.113
33. Zhan, Z., Shen, T.: Development of a prediction model for student teaching satisfaction based on 10 machine learning algorithms. Scientific Reports **15**, 36547 (2025). https://doi.org/10.1038/s41598-025-19039-x